

HEXASYNC GROUP PROJECT

Master Project Documentation

Supervised Learning + Unsupervised Learning
Academic Project Documentation | September 2026

1. Project Overview

This project combines two machine-learning applications developed collaboratively by a six-member team. The supervised-learning component is an Advanced Loan Approval system, while the unsupervised-learning component is a Music Listener Segmentation system. The project demonstrates the complete workflow from dataset preparation and exploratory analysis to model training, validation, deployment, and user-facing prediction.

2. Team Members and Responsibilities

Member	Primary Role	Major Contribution
Mugilan	Loan Approval – ML & Model Development Lead	Loan dataset understanding; preprocessing and feature handling; classifier training and comparison; model evaluation; final model selection; saving model/scaler; ML integration support.
Ashaaz Ahmed Khan	Loan Approval – Dataset & Testing Lead	Dataset generation/handling through dataset.py; data validation; test-case preparation; final testing; checking prediction behaviour and documenting test observations.
Anto Roshan	Loan Approval – Backend & Frontend Developer	FastAPI prediction backend; model/scaler loading; API endpoint; Streamlit user interface; API-to-frontend integration; application testing.
Pavithra	Music Listener Segmentation – Data Analysis & EDA Lead	Dataset inspection and cleaning; missing-value, duplicate and outlier checks; EDA; correlation and distribution analysis; PCA/elbow/silhouette exploration; charts; cleaned dataset and hand-off documentation.
Srimathi	Music Listener Segmentation – Clustering & ML Lead	Feature selection and scaling; K-Means clustering; elbow and silhouette analysis; cluster-center interpretation; model/scaler artifacts; clustered output and segment interpretation.
Nithish	Music Listener Segmentation – Application & UI Lead	Planned application layer using the trained K-Means model and scaler; user input handling; prediction display; segment-name

		presentation; application testing and UI integration.
--	--	---

3. Project A – Advanced Loan Approval (Supervised Learning)

3.1 Objective

To build a supervised machine-learning application that predicts whether a loan application is approved or not from applicant financial and employment attributes, and expose the trained model through an application interface.

3.2 Dataset and Features

The project uses loans.csv with the following input features: income, credit_score, loan_amount, and employment_years. loan_status is the target variable, with 0 representing rejection/not approved and 1 representing approval.

3.3 Data Generation and Validation

Ashaaz prepared the dataset-generation/testing component. The supplied role report describes dataset.py as generating a balanced synthetic loan dataset using credit score, income, employment experience and a debt-burden penalty. The same report describes an eight-profile validation suite covering different financial edge cases.

3.4 Machine-Learning Pipeline

- Load and inspect the loan dataset.
- Separate the four input features from the loan_status target.
- Split the data into training and testing portions using stratification.
- Fit StandardScaler on the training data and transform the test data with the fitted scaler.
- Train and compare suitable supervised classifiers.
- Evaluate predictions using accuracy, precision, recall, F1-score and a confusion matrix.
- Select the final model based on the documented evaluation results and save the trained model and scaler with Joblib.

3.5 Model Validation

The testing contribution includes eight deliberately different borrower profiles, including poor-credit applicants, high earners with weak credit, fresh graduates and high-debt cases. The validation script uses the same feature names and scaler transformation as the training pipeline and uses predict_proba() to inspect prediction confidence.

3.6 Application Architecture

Dataset → preprocessing/scaling → supervised classifier → saved model/scaler → FastAPI backend → Streamlit frontend → approval result.

Anto's application responsibility is to expose the trained model through FastAPI on port 8000 and connect the backend to the Streamlit interface. The application should load the saved artifacts rather than retraining the model.

4. Project B – Music Listener Segmentation (Unsupervised Learning)

4.1 Objective

To discover meaningful listener groups from behavioural music-usage data without a predefined target label, using K-Means clustering.

4.2 Dataset

Pavithra's EDA report documents a 30-listener dataset with four numeric behavioural features: listening_hours_per_week, songs_per_day, skip_rate, and playlist_count. There is no ID, text, or target column.

4.3 EDA and Data Preparation – Pavithra

- Loaded and inspected the dataset shape, data types, missing values and duplicates.
- Verified that all 30 rows were retained; the report records zero missing values, zero duplicate rows and no impossible values.
- Checked outliers using the IQR rule and found no outliers requiring capping.
- Examined distributions with histograms and boxplots.
- Studied feature relationships using correlation analysis and scatter plots.
- Prepared StandardScaler-based clustering inputs and documented the required feature order.
- Performed PCA, elbow and silhouette checks and ran a preliminary K-Means preview.
- Produced charts, findings, data-quality documentation and the cleaned hand-off dataset.

The EDA report records very strong correlations among the four features and notes that the first PCA component explains 96.7% of the variation. It also reports an elbow indication around $k=3$ and silhouette scores of 0.578 for $k=2$ and 0.553 for $k=3$.

4.4 Clustering and Model Development – Srimathi

- Selected the four behavioural features for clustering.
- Applied StandardScaler before K-Means.
- Evaluated candidate cluster counts using elbow and silhouette analysis.
- Selected $k=3$ for the project requirement and supporting analysis.
- Fitted the final K-Means model and examined cluster centres in original units for interpretation.
- Saved model.pkl and scaler.pkl for reuse by the application.
- Prepared clustered listener output and segment interpretations.

The submitted clustering report describes three segments as Casual, Regular and Power Listeners, with cluster-level averages of approximately 3.9, 14.0 and 29.9 listening hours per week respectively. It reports a silhouette score of about 0.54.

Important data-version note: the EDA report describes the source dataset as 30 listeners, while the clustering report contains results for 360 listeners. These figures indicate that the modelling report used a different/larger dataset version or an expanded dataset. Before final submission, the team should verify which dataset version is the official project dataset and regenerate the final model/results consistently from that version.

4.5 Application and UI – Nithish

Nithish is responsible for the application layer that consumes the trained model. The planned flow is to load model.pkl and scaler.pkl, collect the four listener features, apply scaler.transform() using the same feature order as training, call model.predict(), map the cluster to its human-readable segment name, and display the result through the Streamlit interface.

Recommended service separation: Music Listener FastAPI backend on port 8001 so it does not conflict with the Loan Approval API on port 8000.

5. End-to-End System Architecture

Supervised system:

loans.csv → preprocessing → classifier training → model.pkl + scaler.pkl → FastAPI (8000) → Streamlit → Loan Approval Result

Unsupervised system:

music_listeners.csv → Pavithra EDA/cleaning → cleaned dataset → StandardScaler + K-Means → model.pkl + scaler.pkl → FastAPI (8001) → Streamlit → Listener Segment

6. Project Deliverables

Component	Key Files / Output	Responsible Member
Loan dataset	loans.csv, dataset.py	Ashaaz Ahmed Khan
Loan ML	train_model.py, model.pkl, scaler.pkl	Mugilan
Loan validation	test_model.py / test profiles	Ashaaz + Mugilan
Loan backend/UI	main.py, app.py	Anto Roshan
Music EDA	eda.py, cleaned dataset, charts, reports	Pavithra
Music clustering	train_model.py, model.pkl, scaler.pkl, clustered output	Srimathi
Music application	FastAPI integration and Streamlit app	Nithish
Documentation	Individual reports combined into master documentation	Team

7. Collaboration and Handover Process

1. Pavithra hands music_listeners_cleaned.csv to Srimathi. The unscaled cleaned file is the modelling input because the final training pipeline fits and saves its own scaler.
2. Srimathi hands model.pkl and scaler.pkl, together with the exact feature order and cluster-to-segment mapping, to Nithish.
3. Ashaaz supplies the loan dataset and validation/test cases; Mugilan supplies the trained loan model and scaler to Anto.
4. Anto integrates the loan model into FastAPI and Streamlit. Nithish integrates the music model into the music application.
5. Each member commits their work to an individual feature branch before the team merges verified work into main.

8. Technical Practices

- Use the same feature order during training and prediction.
- Fit StandardScaler only on training data or the official clustering dataset, then use transform() for new inputs.
- Do not fit the scaler again inside the prediction application.
- Save trained artifacts with Joblib and load them in the backend/application.
- Keep supervised and unsupervised project files in separate folders to avoid filename conflicts.
- Test the API and UI with normal and edge-case inputs before final demonstration.

9. Limitations

For the music project, Pavithra's report notes that the 30-listener dataset is small and that the four features are extremely correlated, meaning clustering largely represents one overall listening-activity scale. Cluster names are interpretations of K-Means centres rather than labels supplied by the data. The final team should also resolve the dataset-size discrepancy between the EDA report and the clustering report before presenting final results.

10. Conclusion

The HexaSync project demonstrates two complementary machine-learning approaches. The Loan Approval component applies supervised learning to predict an approval outcome, supported by dataset preparation, model development, validation, backend integration and a user interface. The Music Listener Segmentation component applies unsupervised K-Means clustering to discover behavioural listener groups, supported by systematic EDA, cluster analysis, reusable model artifacts and an application layer. Together, the six members contribute across data preparation, analysis, machine learning, testing, backend development, frontend development and documentation, providing an end-to-end academic ML workflow.

11. Final Team Responsibility Summary

Mugilan – Loan ML/model development; Ashaaz Ahemed – loan dataset and testing; Anto Roshan – loan backend and frontend; Pavithra P. – music EDA and data preparation; Srimathi – music clustering/model development; Nithish – music application/UI.